Semantic Similarity Search

Embedding Models, ANN Algorithms & Hybrid Search

MTEB Benchmark | SentenceTransformers | FAISS | OpenAI Embeddings

Overview

Semantic similarity search finds documents, passages, or items that are conceptually related to a query — not just those matching exact keywords. It powers search engines, recommendation systems, RAG pipelines, duplicate detection, and clustering.

1. Embedding Models for Similarity

The quality of similarity search is determined primarily by the quality of the embedding model. Embeddings must be semantically meaningful: similar concepts must be close in the embedding space.

Model	Dim	Context	Best Use
text-embedding-3-large (OpenAI)	3072	8191 tok	General purpose, multilingual
text-embedding-3-small (OpenAI)	1536	8191 tok	Cost-efficient RAG
E5-large-v2 (Microsoft)	1024	512 tok	Open-source, strong benchmarks
BGE-M3 (BAAI)	1024	8192 tok	Multilingual, long-doc
all-MiniLM-L6-v2 (SBERT)	384	256 tok	Fast local inference
voyage-3 (Anthropic)	1024	32000 tok	Claude ecosystem, code+text
CLIP ViT-L (OpenAI)	768	77 tok	Image-text joint embedding
Cohere embed-v3	1024	512 tok	Multilingual semantic search

2. MTEB Benchmark

The Massive Text Embedding Benchmark (MTEB) by Muennighoff et al. (2022) is the standard evaluation suite for embedding models across 56 tasks and 112 languages including retrieval, clustering, reranking, classification, summarization, and pair classification. Models are compared on average NDCG@10 for retrieval tasks.

3. Approximate Nearest Neighbor (ANN) Search

Exact nearest neighbor search requires O(n*d) time per query, which becomes infeasible at scale. ANN algorithms trade recall for speed:

```
Recall@k = |Approximate top-k ∩ True top-k| / k
```

Target recall@10 of >95% is typically acceptable for production RAG systems. HNSW achieves >99% recall@10 at 1000+ QPS on million-vector datasets.

4. Hybrid Search: Dense + Sparse

Dense retrieval excels at semantic similarity but can miss exact keyword matches. Sparse retrieval (BM25) excels at keyword matching but misses synonyms. Hybrid search combines both via Reciprocal Rank Fusion (RRF):

```
RRF_score(d) = sum_i[ 1 / (k + rank_i(d)) ] where k=60
```

5. Reranking

A two-stage pipeline vastly improves precision: (1) retrieve top-100 candidates cheaply with ANN; (2) rerank with a cross-encoder model (e.g., Cohere Rerank, BGE-reranker-large, FlashRank). Cross-encoders jointly encode query+document, achieving much higher accuracy than bi-encoders at the cost of latency.

6. End-to-End Example: Semantic Search with Qdrant

```
from qdrant_client import QdrantClient
from qdrant_client.http.models import Distance, VectorParams, PointStruct
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-large-en-v1.5")
client = QdrantClient(":memory:")  # or host="localhost"

# Create collection
client.create_collection(
    collection_name="docs",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE)
)

# Upsert documents
texts = ["Claude is an AI assistant", "Vector databases store embeddings", ...]
embeddings = model.encode(texts, normalize_embeddings=True)
points = [PointStruct(id=i, vector=emb.tolist(), payload={"text": t})
          for i, (emb, t) in enumerate(zip(embeddings, texts))]
client.upsert(collection_name="docs", points=points)

# Search
query_vec = model.encode(["How does RAG work?"], normalize_embeddings=True)[0]
results = client.search(collection_name="docs", query_vector=query_vec.tolist(), lim
```

```
it=5)
for r in results:
    print(f"Score: {r.score:.3f} | {r.payload['text']}")
```